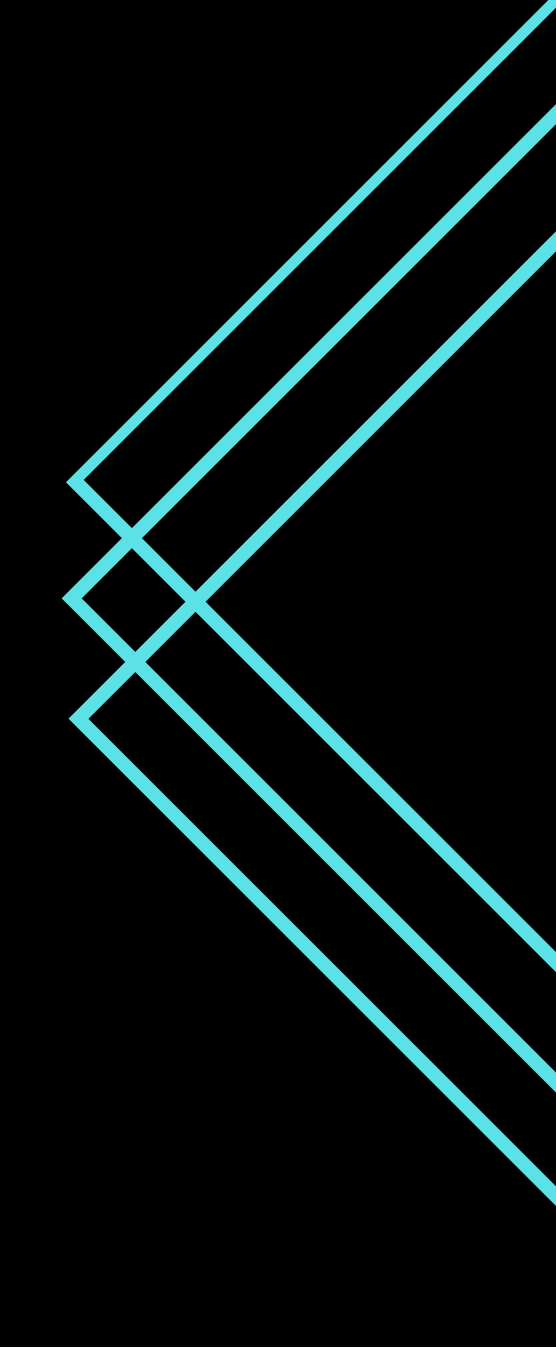
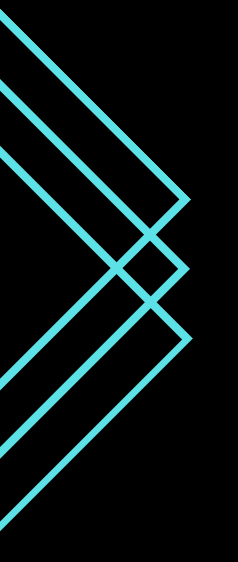




November 2022

SOURCE CODE REVIEW REPORT

ONLINE MANAGEMENT SYSTEM



Prepared by Meryem I.

CONFIDENTIALITY

In no event shall Meryem I. be liable to anyone for special, incidental, collateral or consequential damages arising out of the use of this information. This document contains information, which is confidential and proprietary to Meryem I. and [company]. Extreme care should be exercised before distributing copies of this document, or the extracted contents of this document. Meryem I. is authorizing our point of contact at [company] to view and disseminate this document as he/she sees fit in accordance with [company] data handling policy and procedures. This document should be marked "CONFIDENCIAL" and therefore we suggest that this document be disseminated on a "need to know" basis.

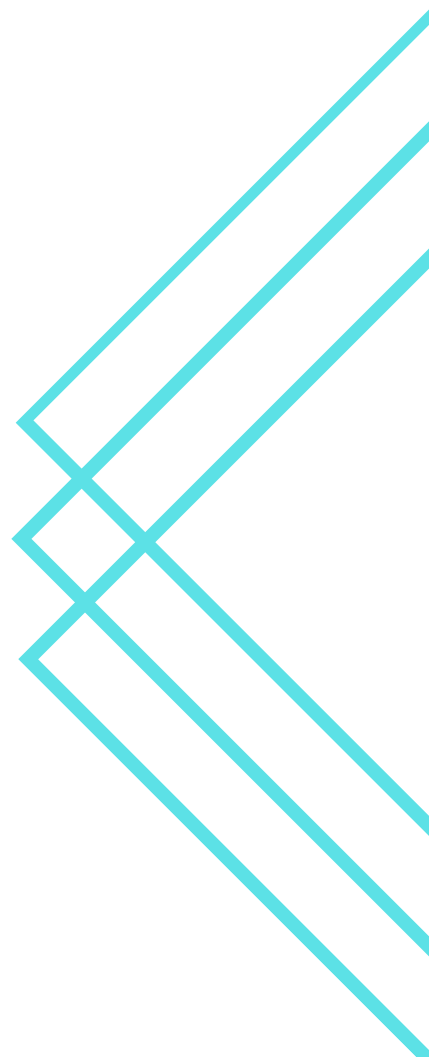
DISCLAIMERS

The information presented in this document is provided as is and without warranty. Source code reviews are a "point in time" analysis and as such it is possible that something in the environment could have changed since the tests reflected in this report were run. Also, it is possible that new vulnerabilities may have been discovered since the tests were run. For this reason, this report should be considered a guide, not a 100% representation of the risk threatening your systems, networks and applications.



Table of Contents

Confidentiality	02
Disclaimers	02
Table of Contents	03
Executive Summary	04
The Code Review Process	05
Finding Details	06
SQL Injection - 1	07
SQL Injection - 2	08
Unrestricted File Upload	09
Insecure Direct Object Reference	10
Cross Site Scripting via SVG Upload	11
Appendix A References	12



Executive Summary

The source code review of the Management System application was completed on September 21, 2021 by Meryem Irmak.

The review was performed on code obtained from website on September 14, 2021, and bundled under the file named management_system_v2.tar.gz.

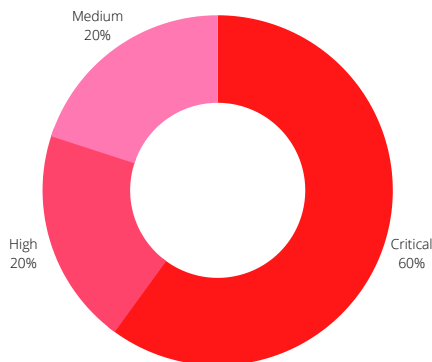
A meeting between Meryem Irmak and [redacted name] was held on September 12, 2021, at which time information about the code structure was presented along with high level overviews of how things like authentication, data validation, and logging were implemented in the code. This information was used by the review team to formulate a plan for the impending review. The actual review involved a manual investigation of the PHP code. The specific findings are presented in the next section.

The assessment was performed from XX/XX/XXXX to XX/XX/XXXX.

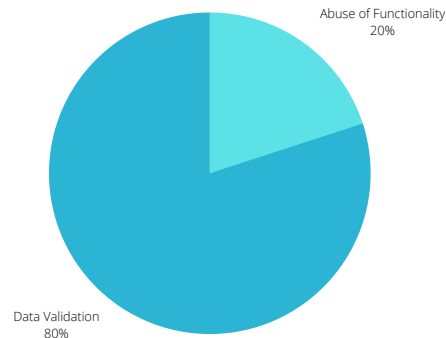
The vulnerabilities have been marked according to the following table:

Severity	Description
Critical	Easy exploitation / High Business Impact
High	Difficult exploitation / High Business Impact
Medium	Difficult exploitation / Low Business Impact
Low	Low and informational level issues

The overall security rating of the application was determined as "**critical**".



Severity Distribution



Vulnerability Class Distribution

The Code Review Process

A Secure Code Review is a specialized task with the goal of identifying types of weaknesses that exist within a given code base. The task involves both manual and automated review of the underlying source code and identifies specific issues that may be representative of broader classes of weakness inherent in the code.

During the actual review, members of a review team review the application code for security problems and categorize the findings based on the weakness categories (e.g., data validation, authorization, etc.). Each finding is assigned a risk rating of Critical, High, Medium or Low. These findings and the broader weakness classes that they represent are presented in this final report that the development team can use as the foundation for improving the overall quality of the code base.

It should be noted that while the review process will be as thorough as possible in finding and reporting security weaknesses, it is not guaranteed to always find every possible weakness. A Secure Code Review is not a silver bullet, but instead is a strong part of an overall risk mitigation program to protect an application.



Finding Details

This section provides details about the specific weaknesses that were found during the review. These details are designed to provide the developers with proof that the stated weaknesses exist as well as to provide examples that the developers can use to find and fix similar areas of the code. As mentioned before, the Secure Code Review does not claim to find every issue; as such the development team should use the information in these findings as an opportunity to improve the entire code base. Just fixing the specific examples identified below will most likely not remove the higher level risks from the application.

Each finding is given a qualitative risk rating assigned by the reviewers at the time of the review. The risk ratings should be considered risks to the application itself. In other words, the risk that the application behavior could be subverted in an unintended way could lead to a possible compromise. This information should then be used by the appropriate teams (developers/management/Information Security) in conjunction with the additional 'big picture' information that they have, to make the appropriate risk mitigation decisions.

SQL Injection

Weakness: <https://cwe.mitre.org/data/definitions/89.html>

Source File: classes/Login.php

Line Number: 21

Risk: Critical

Class: Data Validation

Description: An SQL Injection vulnerability exists in the Login functionality of Learning System v2.0, which allows an attacker to inject their own SQL query. The username parameter from /admin/login.php is vulnerable to SQL injection. An attacker can make an unauthenticated HTTP request with access to "Login" functionality to trigger this vulnerability.

Recommendations: SQL injection can be prevented by using parameterized queries (also known as prepared statements) instead of string concatenation within the query. The code the application uses is vulnerable to SQL injection because the user input is concatenated directly into the query.

SQL Injection

Weakness: <https://cwe.mitre.org/data/definitions/89.html>

Source File: admin/students/manage.php

Line Number: 4

Risk: Critical

Class: Data Validation

Description: An SQL Injection vulnerability exists in the Learning System v2.0, which allows an attacker to inject their own SQL query. The id parameter from /admin/students/manage.php is vulnerable to SQL injection. An attacker can make an unauthenticated HTTP request with this endpoint to trigger this vulnerability.

Recommendations: SQL injection can be prevented by using parameterized queries (also known as prepared statements) instead of string concatenation within the query. The code the application uses is vulnerable to SQL injection because the user input is concatenated directly into the query.

Unrestricted File Upload

Weakness: <https://cwe.mitre.org/data/definitions/434.html>

Source File: classes/Users.php

Line Number: 28-35

Risk: Critical

Class: Data Validation

Description: An Arbitrary File Upload vulnerability exists in Learning System allows the unauthenticated user to conduct remote code execution via classes/Users.php? f=save because .php files can be uploaded to /uploads/ (under the web root).

Recommendations: File types that can be uploaded by the user should be defined as whitelists and uploading files other than these should not be allowed.

Insecure Direct Object Reference

Weakness: <https://cwe.mitre.org/data/definitions/639.html>

Source File: admin/faculty/manage.php

Line Number: 3

Risk: High

Class: Abuse of Functionality

Description: An insecure direct object reference (IDOR) is an access control vulnerability where unvalidated user input can be used for unauthorized access to resources or operations.

Recommendations: The most robust approach to eliminating IDOR vulnerabilities is to ensure proper session management and object-level user access control checks. It should be checked whether the user who wants to perform the operation is an admin.

Cross Site Scripting via SVG Upload

Weakness: <https://cwe.mitre.org/data/definitions/79.html>

Source File: classes/Master.php

Line Number: 462

Risk: Medium

Class: Data Validation

Description: Cross-site scripting (also known as XSS) is a web security vulnerability that allows an attacker to compromise the interactions that users have with a vulnerable application. It allows an attacker to circumvent the same origin policy, which is designed to segregate different websites from each other. Cross-site scripting vulnerabilities normally allow an attacker to masquerade as a victim user, to carry out any actions that the user is able to perform, and to access any of the user's data. XSS works by manipulating a vulnerable website so that it returns malicious JavaScript to users. When the malicious code executes inside a victim's browser, the attacker can fully compromise their interaction with the application.

Recommendations: When checking for user-uploaded files, the image/svg type should also be blocked.

Appendix A References

- Christey, S. (2010). 2010 CWE/SANS Top 25 Most Dangerous Programming Errors. Retrieved from <http://cwe.mitre.org>